Guida completa all'Esercizio 2

CPU RISC-V a ciclo singolo con una nuova istruzione

Metodo di risoluzione e soluzioni dei sei appelli passati

Indice

1 Com'è fatto l'esercizio **3**

2 Il datapath di partenza **3**

2.1 Componenti e funzione . . . 3

2.2 Tabella dei segnali di controllo . . . 4

3 Punto 1: come modificare il datapath **4**

3.1 Il metodo in 6 passi . . . 4

3.2 Catalogo delle modifiche ricorrenti . . . 5

3.3 Cosa consegnare nel disegno . . . 5

4 Punto 2: la codifica in bit **5**

4.1 I formati . . . 5

4.2 La procedura . . . 6

4.3 Tabella dei registri . . . 6

4.4 Opcode . . . 6

5 Punto 3: la durata del ciclo di clock **6**

5.1 Il principio . . . 6

5.2 Come ricostruire il percorso della nuova istruzione . . . 7

5.3 La regola finale . . . 7

5.4 Errori tipici sul tempo . . . 7

6 Punto 4: segnali difettosi e domande teoriche **8**

6.1 Come capire il valore di RegWrite . . . 8

6.2 Variante A: CU difettosa . . . 8

6.3 Variante B: domanda teorica . . . 9

7 Soluzioni dei sei appelli **9**

7.1 Appello 1: `lwig outreg, val1, val2_address` 9
7.2 Appello 2: `bltsafe rd, rs1, rs2` 10
7.3 Appello 3: `jalf rd, rs1, immediate` 11
7.4 Appello 4: `msum a, b, c, d, m` 12
7.5 Appello 5: `mdiff a, b, c, d` 14
7.6 Appello 6: `ajmp res, op1, op2, switch` 15

8 Riepilogo **16**
8.1 Errori da evitare 16
8.2 Come ripassare 17

1 Com'è fatto l'esercizio

In tutti e sei gli appelli l'Esercizio 2 ha la stessa struttura: si parte dal datapath RISC-V a ciclo singolo e si aggiunge una nuova istruzione inventata dal docente.

Punto	Cosa chiede	Tipo di abilità
1	Modificare il datapath e indicare i segnali di controllo	Progettazione
2	Scrivere la word a 32 bit dell'istruzione	Meccanico, punti sicuri
3	Calcolare la durata del ciclo di clock	Meccanico, punti sicuri
4	Varia: codice con CU difettosa, oppure domanda teorica	Ragionamento / teoria

I punti 2, 3 e 4 si risolvono con un metodo preciso. Il punto 1 richiede di aver capito davvero il datapath.

Appello	Nuova istruzione	Punto 4
1	`lwig` (load word if greater, tipo I)	CU difettosa: `RegWrite := MemWrite`
2	`bltsafe` (branch if less than safe, tipo R)	CU difettosa: `RegWrite := Blts`
3	`jalf` (jump and link if divisible by 4, tipo I)	CU difettosa: `RegWrite := MemWrite`
4	`msum` (many sums, tipo R)	Teoria: Genera Cost
5	`mdiff` (many differences, tipo R)	Teoria: Branch e Zero
6	`ajmp` (alu jump, tipo R)	Teoria: Branch e Zero

2 Il datapath di partenza

2.1 Componenti e funzione

- **PC**: contiene l'indirizzo dell'istruzione corrente.
- **Add (PC+4)**: calcola l'indirizzo dell'istruzione successiva.
- **Memoria istruzioni**: dato il PC restituisce la word `Istruzione[31-0]`.
- **Unità di controllo (CU)**: legge `Istruzione[6-0]` (l'opcode) e genera i segnali di controllo (in blu nella figura).
- **Banco registri**: due porte di lettura e una di scrittura.
  - Registro di lettura 1 = `Istruzione[19-15]` (rs1).
  - Registro di lettura 2 = `Istruzione[24-20]` (rs2).
  - Registro di scrittura = `Istruzione[11-7]` (rd).
  - La scrittura avviene solo se `RegWrite`= 1.
- **Genera Cost**: guarda l'istruzione intera e produce l'immediato a 32 bit con estensione del segno, nel formato corretto per il tipo di istruzione.

- **Mux ALUSrc**: sceglie il secondo operando dell'ALU, cioè Dato letto 2 (0) oppure l'immediato (1).
- **ALU e Controllore ALU**: il controllore riceve `ALUOp` (2 bit) più `Istruzione[30,14-12]` (bit 30 e funct3) e genera il codice a 4 bit per l'ALU: `0000` AND, `0001` OR, `0010` somma, `0110` sottrazione. L'ALU restituisce il risultato e il flag **Zero**, che vale 1 se il risultato è 0.
- **Memoria dati**: legge se `MemRead`= 1, scrive se `MemWrite`= 1.
- **Mux MemtoReg**: sceglie il dato da scrivere nei registri, cioè risultato ALU (0) oppure dato letto dalla memoria (1).
- **Add (PC + immediato)**: calcola l'indirizzo di salto.
- **Mux PCSrc**: sceglie il nuovo PC. Il select è `PCSrc` = `Branch` $\wedge$ `Zero`: con 0 si prende PC+4, con 1 si prende l'indirizzo di salto.

2.2 Tabella dei segnali di controllo

Istruzione	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp
R-type (add, sub)	0	0	1	0	0	0	10
lw	1	1	1	1	0	0	00
sw	1	X	0	0	1	0	00
beq	0	X	0	0	0	1	01
I-type (addi)	1	0	1	0	0	0	10

X significa don't care. Questa tabella va imparata a memoria: serve per il punto 1, per il punto 4 e per capire cosa succede nel datapath.

3 Punto 1: come modificare il datapath

3.1 Il metodo in 6 passi

1. **Classifica l'istruzione e mappa i campi.** Chiediti se è di tipo R, I, S o B e dove finiscono i nomi dell'assembly. Per le istruzioni inventate la sintassi non segue l'ordine dei campi: per esempio in `mdiff a, b, c, d` i campi sono a = rs1, b = rs2, c = rd, d = funct7, e il testo lo dice sempre.
2. **Scrivi a parole cosa deve succedere**, passo per passo, nell'ordine del testo.
3. **Per ogni passo chiediti cosa esiste già.** Per esempio "somma rs1 + immediato" è già fatta da ALU e Genera Cost, mentre "somma quattro valori costanti" non esiste e servono sommatori nuovi.
4. **Aggiungi solo i componenti che mancano.** I componenti elementari ammessi sono sommatori, comparatori, MUX, porte logiche (AND, OR, NOT) e shift di quantità costante, che sono semplici fili spostati.
5. **Inserisci i nuovi dati nel datapath con dei MUX.** Non si collegano mai due fili sulla stessa entrata. Ogni punto in cui possono arrivare dati diversi richiede un MUX, il cui select è un nuovo segnale di controllo, di solito chiamato come l'istruzione (`Lwig`, `Blts`, `Jalf`, `Msum`, `Mdiff`, `Ajmp`).
6. **Scrivi la tabella dei segnali** di tutta la CU per la nuova istruzione, con X dove il valore è indifferente.

3.2 Catalogo delle modifiche ricorrenti

1. **Scrivere nei registri un dato nuovo** (PC+4, il risultato di un sommatore, val1 o val2). Si mette un MUX davanti a "Dato da scrivere", oppure si allarga il MUX MemtoReg. Il select è il segnale nuovo. Servono `RegWrite`= 1 e i segnali di memoria a 0.
2. **Scrivere in un registro fisso** (per esempio a0 = x10). Si mette un MUX sull'ingresso "Registro di scrittura" con ingressi `Istruzione[11-7]` e la costante `01010`. Il select è il segnale nuovo.
3. **Cambiare dove va il PC.** Si mette un MUX all'ingresso del PC: da una parte l'uscita del MUX PCSrc esistente, dall'altra il nuovo valore (ALU, sommatore, ecc.). Il select è il segnale nuovo, spesso combinato con altre condizioni.
4. **Condizioni composte.** Si usano porte logiche, per esempio $\texttt{PCSrc} = (\texttt{Branch} \wedge \texttt{Zero}) \vee (\texttt{Blts} \wedge c_1 \wedge c_2)$.
5. **Zero-extension** (estensione senza segno). Non serve nessun componente: si prendono i bit e si aggiungono fili collegati a 0.
6. **Shift di una costante** ($\times 2$, $\times 4$, $\times 8$). Sono solo fili spostati e non costano tempo.
7. **Divisibilità per 4.** Un numero è divisibile per 4 se i suoi due bit meno significativi sono `00`: basta una NOR sui bit 1 e 0.
8. **Condizione "valore $\leq$ 0x7FFFFFFF".** Per un valore a 32 bit non negativo equivale a "il bit 31 è 0", quindi basta un NOT sul bit 31. Rigorosamente si dovrebbe controllare anche il riporto uscente del sommatore; se il PC sta nell'area del programma (valori piccoli) il riporto non può verificarsi.
9. **Condizionare la scrittura.** Se la scrittura dipende da una condizione sui dati, si mette una AND davanti a `RegWrite`: la CU guarda solo l'opcode e non può conoscere i valori.
10. **Confronto tra due valori** ($\geq$, $<$). Si aggiunge un comparatore con le due uscite dei registri, oppure si usa l'ALU in sottrazione. Il comparatore è più semplice da disegnare.

3.3 Cosa consegnare nel disegno

- I componenti nuovi in un colore diverso, con tutti i fili di collegamento disegnati.
- Il nome di ogni segnale nuovo.
- Accanto o sotto, la tabella dei valori di tutti i segnali di controllo per la nuova istruzione.

4 Punto 2: la codifica in bit

4.1 I formati

Tipo R					
funct7	rs2	rs1	funct3	rd	opcode
`[31:25]`	`[24:20]`	`[19:15]`	`[14:12]`	`[11:7]`	`[6:0]`
7 bit	5 bit	5 bit	3 bit	5 bit	7 bit

Tipo I				
imm[11:0]	rs1	funct3	rd	opcode
`[31:20]`	`[19:15]`	`[14:12]`	`[11:7]`	`[6:0]`
12 bit	5 bit	3 bit	5 bit	7 bit

4.2 La procedura

1. Determina il formato (R o I).
2. Associa a ogni campo il suo valore. **Attenzione all'ordine degli operandi nella sintassi assembly**, che può non coincidere con l'ordine dei campi.
3. Converti ogni valore in binario con la larghezza giusta, aggiungendo zeri a sinistra.
4. I campi non usati si mettono a zero, come dice il testo.
5. Concatena i campi, poi raggruppa a 4 bit per ottenere l'esadecimale.

4.3 Tabella dei registri

Registro	**x**	**Registro**	**x**
zero	x0	s0	x8
ra	x1	s1	x9
sp, gp, tp	x2, x3, x4	a0–a7	x10–x17
t0–t2	x5–x7	s2–s11	x18–x27
		t3–t6	x28–x31

4.4 Opcode

L'opcode dato in esadecimale si scrive su 7 bit: `0x3D` = `0111101`, `0x38` = `0111000`, `0x3A` = `0111010`.

5 Punto 3: la durata del ciclo di clock

5.1 Il principio

Il ciclo deve durare quanto l'istruzione più lenta del datapath. Indico con M l'accesso alle memorie, con R l'accesso ai registri e con A le operazioni di ALU e sommatori. MUX, Genera Cost, porte logiche e shift costanti hanno ritardo trascurabile. La **scrittura nei registri conta come un accesso** R.

Istruzione	**Percorso**	**Durata**
R-type	$M + R + A + R$	$M + 2R + A$
lw	$M + R + A + M + R$	$2M + 2R + A$
sw	$M + R + A + M$	$2M + R + A$
beq	$M + R + A$	$M + R + A$

La lw è la più lenta tra le istruzioni di base.

5.2 Come ricostruire il percorso della nuova istruzione

Il percorso non va "scelto" da un elenco: si ricostruisce dalla catena delle dipendenze.

1. **Individua i punti d'arrivo**: scrittura nel banco registri (conta R), scrittura in memoria dati, aggiornamento del PC.
2. **Parti dal fetch.** Ogni istruzione comincia con la lettura dalla memoria istruzioni (M), perché opcode, registri e immediato escono da lì. Il sommatore PC+4 dipende solo dal PC, quindi il suo tempo è nascosto dal fetch.
3. **Per ogni punto d'arrivo risali all'indietro**, chiedendoti "di cosa ho bisogno per scrivere questo?", fino ad arrivare al fetch.
4. **Blocchi in serie: si sommano. Blocchi in parallelo: conta solo il più lento.** Se B ha bisogno dell'uscita di A i tempi si sommano; se A e B partono dagli stessi dati e non dipendono l'uno dall'altro, si conta il più lento.
5. Se un punto d'arrivo ha più catene vince la più lunga; tra i punti d'arrivo vince la catena più lunga dell'istruzione.

Domanda	Se sì	Se no
Servono valori dal banco registri?	aggiungi R	niente R (es. `msum`, valori immediati)
I valori entrano in un'operazione?	aggiungi A	niente A
Ci sono operazioni successive sul risultato?	A per ogni livello in serie	niente
Si accede alla memoria dati?	aggiungi M	niente
Il risultato va nei registri?	aggiungi R finale	niente

5.3 La regola finale

Il ciclo è un massimo

$T_{\text{ciclo}} = \max\big(T_{\text{nuova}},\, T_{\text{lw}}\big)$ con $T_{\text{lw}} = 2M + 2R + A$.
Calcola sempre la catena della nuova istruzione, poi confrontala con la lw e dichiara quale delle due vince.

Non scrivere sempre $2M + 2R + A$

In cinque appelli su sei vince la lw, quindi la risposta coincide con $2M+2R+A$. Nell'Appello 1 però la nuova istruzione (`lwig`) fa un accesso in memoria e poi un confronto in serie, supera la lw e il ciclo diventa 750 ns. Scrivere a priori $2M + 2R + A$ senza confronto fa perdere punti.

5.4 Errori tipici sul tempo

- **Parallelo contro serie.** Lettura registri (50) e sommatore sull'immediato (150) partono insieme: si conta 150, non 200.

- **Cascata contro albero.** Per $a+b+c+d$ con tre sommatori in fila si ha $3A$; con l'albero bilanciato $(a+b)+(c+d)$ si ha $2A$. Conta la struttura disegnata al punto 1.
- **La scrittura nei registri vale** R, non zero.
- **Il PC non richiede la scrittura nei registri**: per un salto il percorso finisce quando il nuovo valore arriva al PC.

6 Punto 4: segnali difettosi e domande teoriche

6.1 Come capire il valore di RegWrite

Per definire una nuova istruzione, la regola è una sola: **RegWrite = 1 se l'istruzione deve scrivere un risultato nel banco registri, altrimenti 0**.

1. Il testo dice "salva/scrive/carica in un registro" (rd, a0, outreg, ...)? Allora `RegWrite` = 1.
2. L'istruzione scrive solo in memoria o cambia solo il PC? Allora `RegWrite` = 0.
3. La scrittura è condizionata a una condizione sui dati? Il segnale della CU è 1 e si inserisce una AND di gating.

Appello	RegWrite	Motivo
1 (lwig)	1	Scrive sempre in rd; cambia solo il dato (val1 o val2), scelto dal MUX, non da RegWrite.
2 (bltsafe)	0	Il campo rd è usato solo come numero per l'offset, non è una destinazione.
3 (jalf)	1, con AND	Si scrive solo se divisibile per 4: $\texttt{RegWrite}' = \texttt{RegWrite} \wedge (\neg \texttt{Jalf} \vee \texttt{Div4})$.
4 (msum)	1	Scrive sempre in a0.
5 (mdiff)	1	Scrive sempre in a0.
6 (ajmp)	1	Scrive sempre in rd, e in più aggiorna il PC.

6.2 Variante A: CU difettosa

Il testo sostituisce RegWrite con un altro segnale e assume che tutti i don't care valgano 0. Per **ogni istruzione** del frammento:

1. **Qual è il valore del segnale che sovrascrive RegWrite?** Dalla tabella dei segnali: `MemWrite` vale 1 solo per `sw`, `Blts` vale 1 solo per `bltsafe`, tutte le altre hanno 0.
2. **Se vale 1, quale registro viene scritto?** Sempre quello indicato da `Istruzione[11-7]`, qualunque sia il tipo di istruzione. Per una `sw` quei bit sono `imm`[4:0].
3. **Quale dato viene scritto?** Quello in uscita dal MUX MemtoReg: con MemtoReg don't care = 0 è il risultato dell'ALU.

Poi si segue l'esecuzione partendo dai valori iniziali (tutti 0 secondo il testo), ricordando che:

- un registro non scritto mantiene il valore iniziale;
- il registro **x0** non può essere modificato;
- i branch si valutano con i valori effettivi dei registri, cioè tenendo conto delle scritture mancate;

- alla fine conviene confrontare con il comportamento corretto, sia come controllo sia per la motivazione.

6.3 Variante B: domanda teorica

Le risposte modello (Genera Cost, Branch e Zero) sono nelle soluzioni degli Appelli 4, 5 e 6.

7 Soluzioni dei sei appelli

7.1 Appello 1: `lwig outreg, val1, val2_address`

Cosa fa. Tipo I con rd = outreg, rs1 = val1, imm = val2_address.

1. Legge val1 da rs1.
2. Estende il segno dell'immediato.
3. Calcola l'indirizzo $\text{imm}_{32} \times 4 + \texttt{0x10010000}$ e legge la word val2 dalla memoria dati.
4. Confronta val1 con val2.
5. Scrive in rd il maggiore dei due.

Modifiche al datapath.

1. Genera Cost estende l'immediato come per una lw.
2. Uno shift left di 2 (fili) fa $\text{imm}_{32} \times 4$.
3. Un **sommatore nuovo** somma il risultato alla costante `0x10010000`.
4. Un **MUX nuovo sull'indirizzo della memoria dati**: ingresso 0 = risultato ALU, ingresso 1 = uscita del sommatore nuovo. Select = `Lwig`.
5. Un **comparatore $\geq$ con segno** riceve Dato letto 1 (val1) e Dato letto della memoria (val2).
6. Un **MUX nuovo** M3: ingresso 0 = dato letto dalla memoria (val2), ingresso 1 = Dato letto 1 (val1). Select = `Lwig` $\wedge$ `GE`. La sua uscita va all'ingresso 1 del MUX MemtoReg esistente. Con `Lwig` = 0 (lw normale) passa sempre la memoria, quindi la lw non cambia comportamento.

Segnali.

`Lwig`	`RegWrite`	`MemRead`	`MemWrite`	`MemtoReg`	`Branch`	`ALUSrc`	`ALUOp`
1	1	1	0	1	0	X	XX

L'ALU non viene usata: l'indirizzo lo calcola il sommatore nuovo.

Codifica di `lwig t4, s4, 0x0203`. Tipo I: imm = `0x203`, rs1 = s4 = x20, funct3 inutilizzato = `000`, rd = t4 = x29, opcode = `0x3D`.

imm[11:0]	rs1	funct3	rd	opcode
001000000011	10100	000	11101	0111101

Word: `0010 0000 0011 1010 0000 1110 1011 1101` = **0x203A0EBD**.

Tempo ($M = 200$, $R = 50$, $A = 150$).

- lw: $200 + 50 + 150 + 200 + 50 = 650$ ns.
- lwig: fetch 200; sommatore dell'indirizzo 150 (in parallelo con la lettura registri da 50, quindi conta 150); memoria dati 200; comparatore 150 (lo conto come operazione ALU); scrittura registri 50.

$$200 + 150 + 200 + 150 + 50 = 750 \text{ ns}.$$

Ciclo di clock

750 ns. Qui la nuova istruzione allunga il percorso critico, perché il confronto deve aspettare val2 che arriva dalla memoria.

Punto 4 (`RegWrite` := `MemWrite`). Nel frammento non c'è nessuna `sw`, quindi `MemWrite`= 0 per tutte le istruzioni e nessun registro viene mai scritto.

Istruzione	MemWrite	RegWrite	Effetto
`addi s0, s0, 0x8004`	0	0	s0 resta 0
`beq s1, s0, Out`	0	0	$0 = 0$, salta a Out
`add a7, zero, s1`	0	0	a7 resta 0
`addi a7, a7, 0x8002`	0	0	a7 resta 0

La `jal x0, Exit` viene saltata dal branch. **a7** $= 0$. Con la CU corretta s0 diventerebbe `0x8004`, il branch non verrebbe preso e a7 varrebbe `0x8002`.

7.2 Appello 2: `bltsafe rd, rs1, rs2`

Cosa fa. Tipo R. Prende i 5 bit di rd con zero-extension (rd_{32}) e salta a $\text{PC} + \text{rd}_{32}$ se e solo se rs1 $<$ rs2 **e** $\text{PC} + \text{rd}_{32} \leq$ `0x7FFFFFFF`.

Modifiche al datapath.

1. Genera Cost, per questo opcode, produce `Istruzione[11-7]` con zero-extension (il testo lo permette).
2. Il sommatore PC + immediato esistente calcola $\text{PC} + \text{rd}_{32}$.
3. Un **comparatore $<$ con segno** riceve Dato letto 1 e Dato letto 2.
4. Un **inverter** sul bit 31 dell'uscita Somma del sommatore: per un valore non negativo "$\leq$ 0x7FFFFFFF" equivale a "bit 31 $= 0$".
5. Una **AND a tre ingressi**: `Blts` $\wedge$ (rs1 $<$ rs2) $\wedge \neg$Somma[31].
6. Una **OR** unisce questa condizione al vecchio `Branch`$\wedge$`Zero`. Il select del MUX PCSrc diventa

$$\texttt{PCSrc} = (\texttt{Branch} \wedge \texttt{Zero}) \vee (\texttt{Blts} \wedge LT \wedge \neg S_{31}).$$

Segnali.

Blts	Branch	RegWrite	MemRead	MemWrite	MemtoReg	ALUSrc	ALUOp
1	0	0	0	0	X	X	XX

Codifica di `bltsafe 16, 19, 3`. rd = 16, rs1 = 19, rs2 = 3. Tipo R con funct7 = 0, funct3 = 0, opcode = `0x38`.

funct7	rs2	rs1	funct3	rd	opcode
0000000	00011	10011	000	10000	0111000

Word: `0000 0000 0011 1001 1000 1000 0011 1000` = **0x00398838**.

Tempo ($M = 200$, $R = 50$, $A = 100$).

- lw: $200 + 50 + 100 + 200 + 50 = 600$ ns.
- bltsafe: fetch 200, poi in parallelo lettura registri più comparatore ($50 + 100 = 150$) e sommatore del PC (100): conta 150. Totale 350 ns.

Ciclo di clock

600 ns, determinato dalla lw. La nuova istruzione non allunga il percorso critico.

Punto 4 (`RegWrite` := `Blts`). Conta solo la `bltsafe`, che ha `Blts` = 1.

Istruzione	Blts	RegWrite	Effetto
`addi s0, zero, 2`	0	0	s0 resta 0
`addi s1, zero, 1`	0	0	s1 resta 0
`bltsafe s0, s1, 5`	1	1	scrive in `Istruzione[11-7]` = rd = **s0**

La domanda chiede s1. La `bltsafe` scrive in s0 (il campo rd), non in s1, e le due `addi` non hanno scritto nulla. Quindi **s1** = 0. Con la CU corretta sarebbe 1. Il trucco è chiedersi sempre *quale* registro viene scritto, non solo *se* viene scritto.

7.3 Appello 3: `jalf rd, rs1, immediate`

Cosa fa. Tipo I. Se imm_{32} è divisibile per 4, allora rd $\leftarrow$ PC + 4 e PC $\leftarrow$ rs1 + imm_{32}.

Modifiche al datapath.

1. Genera Cost estende l'immediato come per un'istruzione I-type.
2. L'ALU, con `ALUSrc` = 1 e `ALUOp` = 00, calcola rs1 + imm_{32} (come per una lw).
3. Una **NOR** sui bit 1 e 0 dell'immediato (`Istruzione[21:20]`) produce `Div4`.
4. Un **MUX nuovo sull'ingresso del PC**: ingresso 0 = uscita del MUX PCSrc esistente, ingresso 1 = risultato ALU. Select = `Jalf` $\wedge$ `Div4`.
5. Un **MUX nuovo davanti a "Dato da scrivere"**: ingresso 0 = uscita MemtoReg, ingresso 1 = uscita del sommatore PC+4. Select = `Jalf`.
6. Una **AND sul RegWrite**: $\texttt{RegWrite}' = \texttt{RegWrite} \wedge (\neg\texttt{Jalf} \vee \texttt{Div4})$.

Segnali.

Jalf	RegWrite	MemRead	MemWrite	MemtoReg	Branch	ALUSrc	ALUOp
1	1 (gated da Div4)	0	0	X	0	1	00

Codifica di `jalf t4, s4, 0x203`. Tipo I con gli stessi valori dell'Appello 1: imm = `0x203`, rs1 = x20, funct3 = `000`, rd = x29, opcode = `0x3D`. Il risultato è identico: **0x203A0EBD**.

Tempo ($M = 200$, $R = 50$, $A = 150$).

- lw: 650 ns.
- jalf, aggiornamento del PC: $200 + 50 + 150 = 400$ ns.
- jalf, scrittura di PC+4: $200 + 150 + 50 = 400$ ns.

Ciclo di clock

650 ns, ancora la lw.

Punto 4 (`RegWrite` := `MemWrite`). Qui `MemWrite` vale 1 solo per la `sw`.

Istruzione	MemWrite	RegWrite	Effetto
`addi s0, s0, 0x4`	0	0	s0 resta 0
`sw t0, 0(s0)`	1	1	scrive nel registro `Istruzione[11-7]`
`beq s1, s0, Out`	0	0	0 = 0, salta a Out
`add a7, zero, s1`	0	0	a7 resta 0
`addi a7, a7, 0x8002`	0	0	a7 resta 0

Per la `sw` i bit [11:7] non contengono rd ma `imm`[4:0] (nel formato S l'immediato è spezzato). Qui l'immediato è 0, quindi quei bit valgono `00000` e il registro scritto è **x0**, che non cambia mai. **a7** = 0. Con la CU corretta sarebbe `0x8002`.

Variante da controllare

Con un immediato diverso da 0 il risultato cambia: per esempio `sw t0, 8(s0)` ha `imm[4:0]` = `01000`, cioè x8 = s0, e la `sw` scriverebbe davvero in s0. Controlla sempre l'immediato della `sw`.

7.4 Appello 4: `msum a, b, c, d, m`

Cosa fa. Tipo R con a = rs1, b = rs2, c = rd, d = funct7, m = funct3. I valori sono **immediati** (letti dall'istruzione, non dal banco registri) e senza segno. Scrive in a0 il valore $(a+b+c+d) \times 2^m$, con $m =$ `funct3[1:0]`. Il PC diventa PC+4.

Modifiche al datapath.

1. **Zero-extension** (fili a 0) di `Istruzione[19:15]` (a), `[24:20]` (b), `[11:7]` (c), `[31:25]` (d). Genera Cost non si può usare.

2. **Sommatore 1** per $a+b$ e **sommatore 2** per $c+d$, in parallelo.
3. **Sommatore 3** per $(a+b)+(c+d)$.
4. **Shift con MUX 4:1**: gli ingressi sono S, $S \ll 1$, $S \ll 2$, $S \ll 3$ (shift costanti, solo fili); il select è `Istruzione[13:12]`. Il bit 14 di funct3 è ignorato.
5. Un **MUX nuovo sul registro di scrittura**: ingresso 0 = `Istruzione[11-7]`, ingresso 1 = costante `01010` (x10 = a0). Select = `Msum`.
6. Un **MUX nuovo davanti a "Dato da scrivere"**: ingresso 0 = uscita MemtoReg, ingresso 1 = uscita dello shift. Select = `Msum`.

Segnali.

Msum	RegWrite	MemRead	MemWrite	MemtoReg	Branch	ALUSrc	ALUOp
1	1	0	0	X	0	X	XX

Il PC si aggiorna con PC+4 perché `Branch` = 0.

Codifica di `msum 2, 3, 10, 0x7f, 2`. a = 2 (rs1), b = 3 (rs2), c = 10 (rd), d = `0x7F` (funct7), m = 2 (funct3), opcode = `0x3A`.

funct7	rs2	rs1	funct3	rd	opcode
1111111	00011	00010	010	01010	0111010

Word: `1111 1110 0011 0001 0010 0101 0011 1010` = **0xFE31253A**.

Tempo ($M = 200$, $R = 50$, $A = 150$).

- lw: 650 ns.
- msum: fetch 200; nessuna lettura registri (i valori sono immediati); due livelli di sommatori in cascata (150 + 150); shift trascurabile; scrittura registri 50. Totale $200 + 300 + 50 = 550$ ns.

Ciclo di clock

650 ns.

Struttura dei sommatori

Con tre sommatori in cascata $((a+b)+c)+d$ il percorso sarebbe $200+450+50 = 700$ ns e il ciclo diventerebbe 700 ns. Con l'albero bilanciato la nuova istruzione non cambia il ciclo.

Punto 4 (teoria): come funziona Genera Cost. È un blocco combinatorio che riceve l'istruzione a 32 bit e produce l'immediato a 32 bit con estensione del segno. Riconosce il formato dall'opcode (bit [6:0]) e per ogni formato prende i bit dell'immediato da posizioni diverse.

Tipo	Immediato	Da dove prende i bit
R	nessuno	l'uscita non viene usata
I	imm[11:0]	`inst[31:20]`
S	imm[11:0]	imm[11:5] = `inst[31:25]`, imm[4:0] = `inst[11:7]`
B (SB)	imm[12:1], bit 0 = 0	imm[12] = `inst[31]`, imm[11] = `inst[7]`, imm[10:5] = `inst[30:25]`, imm[4:1] = `inst[11:8]`
U	imm[31:12], bassi 12 bit = 0	`inst[31:12]`
J (UJ)	imm[20:1], bit 0 = 0	imm[20] = `inst[31]`, imm[10:1] = `inst[30:21]`, imm[11] = `inst[20]`, imm[19:12] = `inst[19:12]`

Per I, S, B e J i bit superiori si riempiono con il **segno, che è sempre** `inst[31]`: è una scelta di progetto di RISC-V che rende l'estensione veloce.

Internamente non c'è quasi logica: ogni formato è un semplice rimescolamento di fili (cablaggio fisso) e un **MUX comandato dall'opcode** sceglie quale cablaggio mandare in uscita. Negli immediati B e J il bit 0 è sempre 0 perché le istruzioni sono allineate a 2 byte: l'immediato è già "moltiplicato per 2", quindi il sommatore PC + imm non richiede uno shift aggiuntivo.

7.5 Appello 5: `mdiff a, b, c, d`

Cosa fa. Come `msum`, ma a0 $\leftarrow (a+b)-(c+d)$. Valori immediati senza segno (a = rs1, b = rs2, c = rd, d = funct7).

Modifiche al datapath.

1. Zero-extension di a, b, c, d (fili a 0). Genera Cost non si può usare.
2. **Sommatore 1** per $a+b$ e **sommatore 2** per $c+d$, in parallelo.
3. Un **sottrattore** (sommatore con ingresso invertito e riporto in ingresso = 1) per $(a+b)-(c+d)$. Lavorando a 32 bit con zero-extension, il risultato è in complemento a 2 e può essere negativo senza problemi.
4. Un **MUX sul registro di scrittura** (ingresso 0 = `Istruzione[11-7]`, ingresso 1 = `01010`), select = `Mdiff`.
5. Un **MUX davanti a "Dato da scrivere"** (ingresso 0 = uscita MemtoReg, ingresso 1 = uscita del sottrattore), select = `Mdiff`.

Segnali.

Mdiff	RegWrite	MemRead	MemWrite	MemtoReg	Branch	ALUSrc	ALUOp
1	1	0	0	X	0	X	XX

Codifica di `mfidd 4, 3, 10, 0x7f`. Nel testo c'è un refuso (`mfidd`): si intende `mdiff`. a = 4 (rs1), b = 3 (rs2), c = 10 (rd), d = `0x7F` (funct7), funct3 inutilizzato = `000`, opcode = `0x3A`.

funct7	rs2	rs1	funct3	rd	opcode
1111111	00011	00100	000	01010	0111010

Word: `1111 1110 0011 0010 0000 0101 0011 1010` = **0xFE32053A**.

Tempo ($M = 200$, $R = 50$, $A = 150$).

- lw: 650 ns.
- mdiff: 200 (fetch) +150 (somme in parallelo) +150 (sottrazione) +50 (scrittura) = 550 ns.

Ciclo di clock

650 ns.

Punto 4 (teoria): Branch e Zero.

- **Zero** è un'uscita dell'ALU e vale 1 se il risultato dell'operazione è 0.
- **Branch** è un segnale della CU, a 1 solo per le istruzioni di salto condizionato.
- `PCSrc` = `Branch` $\wedge$ `Zero` pilota il MUX che sceglie il nuovo PC: con 0 si prende PC+4, con 1 si prende PC + immediato, calcolato dal sommatore dedicato.
- **beq**: la CU imposta `ALUOp` = 01 (sottrazione) e `Branch` = 1. L'ALU calcola rs1 − rs2; se sono uguali `Zero` = 1 e il salto avviene.
- Le **altre istruzioni** hanno `Branch` = 0: anche se per caso Zero vale 1 (per esempio una add che dà 0) il PC prosegue con PC+4. Branch serve proprio a "mascherare" Zero.
- Nel datapath di partenza esiste solo beq; per bne servirebbe invertire Zero.

7.6 Appello 6: `ajmp res, op1, op2, switch`

Cosa fa. Tipo R con res = rd, op1 = rs1, op2 = rs2, switch = funct3. Legge op1 e op2 dal banco registri. Se `funct3[0]`= 0 calcola op1 + op2, altrimenti op1 − op2. Salva il risultato in rd e **aggiorna il PC con lo stesso risultato**.

Modifiche al datapath.

1. Si usa l'ALU esistente con `ALUSrc` = 0 (secondo operando = Dato letto 2).
2. Il controllo dell'ALU deve dipendere da `funct3[0]`. Un **MUX nuovo sull'uscita del Controllore ALU**: ingresso 0 = uscita normale del controllore, ingresso 1 = {0, `Istruzione[12]`, 1, 0}. Select = `Ajmp`. Se `Istruzione[12]`= 0 si ottiene `0010` (somma), se vale 1 si ottiene `0110` (sottrazione).
3. Il risultato dell'ALU va già al MUX MemtoReg (ingresso 0) e quindi al banco registri, con `RegWrite` = 1.
4. Un **MUX nuovo sull'ingresso del PC**: ingresso 0 = uscita del MUX PCSrc, ingresso 1 = risultato ALU. Select = `Ajmp`.

Segnali.

Ajmp	RegWrite	MemRead	MemWrite	MemtoReg	Branch	ALUSrc	ALUOp
1	1	0	0	0	0	0	XX

Il controllo ALU è forzato dal MUX nuovo, quindi `ALUOp` è indifferente.

Codifica di `ajmp 10, 0x1A, 21, 0x3`. rd = 10, rs1 = `0x1A` = 26, rs2 = 21, funct3 = 3, funct7 = 0, opcode = `0x3A`.

funct7	rs2	rs1	funct3	rd	opcode
0000000	10101	11010	011	01010	0111010

Word: `0000 0001 0101 1101 0011 0101 0011 1010` = **0x015D353A**.

Tempo ($M = 200$, $R = 50$, $A = 150$).

- lw: 650 ns.
- ajmp: fetch 200, lettura registri 50, ALU 150: il PC è pronto a 400 ns; la scrittura nei registri arriva a 450 ns.

Ciclo di clock

650 ns.

Punto 4 (teoria): Branch e Zero. La risposta è la stessa dell'Appello 5, completata con l'effetto sul PC: se `Branch` $\wedge$ `Zero` = 1 il nuovo PC è PC + immediato, altrimenti PC+4. Per `ajmp` il salto non passa da qui, perché il PC viene scritto dal MUX nuovo con il risultato dell'ALU.

8 Riepilogo

Appello	Istruzione	Codifica	Ciclo	Punto 4
1	lwig	`0x203A0EBD`	750 ns	a7 = 0
2	bltsafe	`0x00398838`	600 ns	s1 = 0
3	jalf	`0x203A0EBD`	650 ns	a7 = 0
4	msum	`0xFE31253A`	650 ns	teoria: Genera Cost
5	mdiff	`0xFE32053A`	650 ns	teoria: Branch e Zero
6	ajmp	`0x015D353A`	650 ns	teoria: Branch e Zero

8.1 Errori da evitare

1. **Confondere l'ordine degli operandi** dell'assembly con l'ordine dei campi nella word.
2. **Dimenticare che la lw è l'istruzione più lenta**: il ciclo è il massimo tra la nuova istruzione e la lw.

3. **Non contare la scrittura nei registri** come un accesso R.
4. **Sommare operazioni che avvengono in parallelo**: si sommano solo quelle in cascata.
5. **Collegare due fili sulla stessa entrata** invece di usare un MUX.
6. **Dimenticare di fissare i segnali non usati**: scrivi X, e nel punto 4 ricorda che i don't care valgono 0.
7. **Dimenticare x0** nei frammenti con CU difettosa.
8. **Sbagliare i numeri dei registri**: s0 = x8, s1 = x9, t4 = x29, s4 = x20, a0 = x10.
9. **Dimenticare il gating**: se una scrittura o un salto sono condizionati serve una porta logica sul segnale.
10. **Dare il tempo senza spiegare i calcoli**: il testo chiede sempre di spiegare, quindi scrivi il percorso con i numeri.

8.2 Come ripassare

1. Impara a memoria la tabella dei segnali di controllo e i percorsi di R, lw, sw, beq.
2. Per ogni appello, prima di guardare la soluzione, prova a rispondere da solo ai quattro punti e poi confronta.
3. Allenati a disegnare il datapath da zero, con i MUX nuovi, almeno una volta.
4. Fai la codifica in bit senza guardare gli appunti: con la pratica richiede pochi minuti e sono punti sicuri.